Sentinela del Norte

TC2038: Análisis y Diseño de Algoritmos Avanzados (Grupo 603)

Documento de reflexión: modelos, correctitud, experimentos y aprendizajes.

Versión final (Post-revisión docente) — 22 de octubre de 2026

El Equipo Detrás del Proyecto

Integrante	Matrícula	Responsabilidad Principal
Gael Alexander Zaragoza Torres	A01385956	Arquitectura del ejecutable, protocolo JSON, MST y TSP
Damián Alejandro Gómez Mena	A01385923	Flujo máximo, Voronoi y pruebas de estrés
David Rangel Monsivais	A01385671	Algoritmos de cadenas y aplicación SENTINEL

Resumen del Proyecto: Más allá del código

Para este reto, decidimos ir un paso más allá de entregar una simple colección de scripts sueltos. Construimos un sistema unificado: un único ejecutable en `C++17` capaz de resolver siete problemas algorítmicos distintos sobre la red troncal europea GÉANT, comunicándose íntegramente mediante un protocolo JSON. Sobre esta base sólida, desarrollamos una aplicación interactiva que permite ejecutar los modelos sobre datos reales provenientes de SNDlib y visualizar los resultados en tiempo real.

Para cada desafío técnico, no nos conformamos con una sola solución; implementamos dos aproximaciones distintas. Justificamos nuestra elección basándonos en mediciones empíricas exhaustivas de complejidad temporal y espacial, llevando el sistema al límite con instancias de hasta 200,000 nodos y un millón de aristas.

Tras iterar el diseño con la retroalimentación de nuestro profesor, el sistema alcanzó una calificación perfecta (100/100) en el evaluador en línea en sus tres modos de tiempo (lite, normal y huge). Además, respondimos a la petición de enriquecer el gemelo digital incorporando una interfaz persistente con mapas, visualización granular del flujo máximo enlace por enlace y una simulación dinámica estilo *Packet Tracer* para observar cómo se propaga un mensaje ante fallas críticas en la red.

1. Arquitectura: El Sistema y su Modelo Conceptual

Desde el inicio comprendimos que el verdadero reto no era solo algorítmico, sino de **integración**. Teníamos cuatro áreas (infraestructura, mantenimiento, capacidad y ciberseguridad) resolviendo preguntas sobre la misma red, habitualmente con herramientas separadas. Nuestra visión fue colocar todos los algoritmos detrás de una única interfaz de texto y construir el ecosistema visual sobre ella.

EL CONTRATO: UN JSON ENTRA, UN JSON SALE

Tomamos una decisión arquitectónica radical para un proyecto de este tipo: el núcleo del ejecutable leería un objeto JSON por la entrada estándar y escribiría otro por la salida. El campo `mode` determina el problema, y el campo `input` inyecta los datos. Por ejemplo:

```
{"mode":"mst", "input":{"edges": [[0,1, 4], [0,2,1], [1,2,2]]}}
```

Esta decisión en apariencia menor fue la columna vertebral del sistema y nos brindó tres ventajas gigantescas:

- **Desacoplamiento total:** Los algoritmos pueden ejecutarse desde una terminal, un script de automatización o la aplicación web sin alterar una sola línea de código.
- **Pruebas económicas:** Validar cientos de casos aleatorios tomaba apenas segundos, comparando nuestra salida en C++ contra scripts de fuerza bruta en Python.
- **Tolerancia a fallos:** Si una entrada es inválida, el programa devuelve `{"error": "..."}` en lugar de abortar estrepitosamente. Evitamos por completo las excepciones (incompatibles con el evaluador WebAssembly), gestionando los errores mediante booleanos y paso por referencia.

> *Decidimos no incrustar los algoritmos directamente en la interfaz. Mantener una "frontera de texto" nos obligó a escribir un núcleo en C++ robusto y puramente algorítmico, validable matemáticamente en aislamiento.*

2. Los Datos: Separando la Realidad de la Simulación

Era crucial para nosotros distinguir claramente entre los datos de infraestructura reales y nuestras construcciones para simulación.

Dato Real (SNDlib - geant.txt)	Cantidad	Detalle en el modelo
Nodos	22	Puntos de presencia europeos más Nueva York, con coordenadas exactas.
Enlaces físicos	36	Cada enlace cuenta con un módulo de 40,000 Mbit/s y su costo asociado.
Demandas de tráfico	462	Matriz dirigida de tráfico pico en Mbit/s entre pares de nodos.

Un hallazgo interesante: Al calcular las distancias notamos que el "costo" asignado a cada enlace en SNDlib corresponde casi exactamente a su longitud física en kilómetros (razón entre 0.996 y 1.001). Por ende, cuando nuestra aplicación calcula el costo del MST (16,246), el usuario lo puede interpretar directamente como *kilómetros de fibra óptica*.

La única simulación (Firmas): Como los archivos del reto no estaban disponibles, generamos datos sintéticos transformando la matriz de tráfico real en transmisiones hexadecimales. Es vital notar que

3. El Arsenal Algorítmico

No nos bastaba con entregar algoritmos que funcionaran; queríamos entender *por qué* una técnica era superior a otra en nuestro dominio de datos. A continuación, compartimos nuestra aproximación humana y técnica a cada uno de los problemas del reto.

3.1 Despliegue Óptimo de la Red (MST)

El problema: Conectar todas las centrales europeas gastando lo menos posible en instalación de fibra.

Nuestra elección: `Kruskal + Union-Find`

Aunque implementamos tanto Kruskal como Prim, nos decantamos por Kruskal. GÉANT es una red dispersa (apenas 36 aristas para 22 nodos). Kruskal procesa directamente la lista de aristas sin necesidad de armar complejas listas de adyacencia. Verificamos su correctitud comprobando automáticamente que el árbol resultante siempre tuviera exactamente $n - 1$ aristas.

Técnica	Tiempo	Espacio	Nuestra observación
Kruskal (Union-Find)	$O(E \log E)$	$O(V + E)$	Ideal para el grafo conexo y disperso de la red real.
Prim (Cola prioridad)	$O(E \log V)$	$O(V + E)$	Mejor en teoría para redes densas, pero excesivo aquí.

3.2 Ruta Óptima de Mantenimiento (TSP)

El problema: Diseñar el recorrido de menor costo para una cuadrilla que debe visitar cada nodo y regresar a casa.

Nuestra elección: `Híbrida (Held-Karp o 2-opt)`

Este es el único problema donde el código *toma decisiones por sí solo* dependiendo del tamaño de la entrada. Si la red tiene hasta 16 nodos, usamos programación dinámica exacta (Held-Karp). Más allá de eso, la memoria se desborda, por lo que el sistema cambia automáticamente a una heurística de vecino más cercano refinada iterativamente con el algoritmo 2-opt. Como la red original no es un grafo completo, aplicamos Floyd-Warshall primero para permitir rutas a través de nodos intermedios (¡nadie quiere construir nueva fibra solo para que pase el de mantenimiento!).

Técnica	Tiempo	Espacio	Nuestra observación
Held-Karp	$O(n^2\ 2^n)$	$O(n\ 2^n)$	Exacto, pero la RAM explota al pasar los 16 nodos.
2-opt sobre Vecino	$O(n^2)$/it.	$O(n^2)$	Aproximación sólida (desviación < 7%) para redes grandes.

3.3 Flujo Máximo de Información

El problema: Determinar el volumen máximo de información que puede viajar entre un origen y un destino sin saturar la red.

Nuestra elección: Algoritmo de Dinic

Comparamos Edmonds-Karp con Dinic. Aunque Edmonds-Karp es intuitivo, Dinic organiza la búsqueda mediante grafos por niveles (BFS) y satura caminos completos de una sola pasada (DFS). Para la aplicación visual, añadimos un pase extra para extraer el flujo que pasa exactamente por cada arista y pintarlo en pantalla. Además, calculamos el **corte mínimo** comprobando qué nodos quedan accesibles en la red residual final. La matemática no falla: nuestras pruebas verifican implacablemente que la suma de capacidades de este cuello de botella iguale siempre al flujo máximo.

Técnica	Tiempo	Espacio	Nuestra observación
Dinic	$O(V^2 E)$	$O(V + E)$	Su organización por fases satura la red mucho más rápido.
Edmonds-Karp	$O(V E^2)$	$O(V + E)$	Sirvió como excelente referencia de validación empírica.

3.4 Cobertura Geoespacial (Diagramas de Voronoi)

El problema: Definir qué zonas geográficas son atendidas por cada central (la más cercana).

Nuestra elección: Recorte por Semiplanos

La celda de un sitio inicia abarcando todo el dominio y se va esculpiendo (recortando) con las mediatrices de los sitios adyacentes. Experimentamos con una técnica de ordenamiento previo ("vecinos ordenados") esperando optimizar los recortes, pero descubrimos algo fascinante: el recorte directo era más rápido. Validamos nuestro código con una prueba hermosa y sencilla: si sumamos el área de todas las celdas generadas, el resultado debe ser *exactamente* igual al área del rectángulo original.

Técnica	Tiempo	Espacio	Nuestra observación
Recorte por Semiplanos	$O(n^2 k)$	$O(n k)$	Simple, elegante y, sorpresivamente, la más eficiente en la práctica.
Vecinos ordenados	$O(n^2 \log n)$	$O(n k)$	El costo de ordenar superó los beneficios del recorte anticipado.

3.5 Detección de Firmas de Malware (Búsqueda de Patrones)

Nuestra elección: Algoritmo KMP (Knuth-Morris-Pratt)

Al buscar fragmentos maliciosos en transmisiones, podíamos usar fuerza bruta cruzando los dedos para no encontrar el peor caso. KMP fue la elección obvia para nosotros porque construye una tabla de fallos que le dice al algoritmo cómo "saltar" posiciones de forma inteligente, garantizando que el texto principal jamás se recorra hacia atrás. Consideramos Aho-Corasick, pero al buscar solo tres firmas, la complejidad de armar su autómata no justificaba el esfuerzo de ingeniería.

Técnica	Tiempo	Espacio
KMP	$O(n + m)$	$O(m)$ auxiliar
Fuerza bruta	$O(n \cdot m)$	$O(1)$

3.6 Análisis de Código Espejeado (Palíndromo más largo)

Nuestra elección: Algoritmo de Manacher

Un problema clásico de cadenas donde expandir desde cada centro cuesta tiempo cuadrático. Manacher fue una revelación para el equipo: al intercalar caracteres separadores, trata palíndromos pares e impares por igual. Más importante aún, explota la simetría: si estamos dentro de un palíndromo conocido, asume el radio espejo en vez de recalcular a ciegas. Un recorrido brillante, estrictamente lineal de izquierda a derecha.

Técnica	Tiempo	Espacio
Manacher	$O(n)$	$O(n)$
Expandir por centros	$O(n^2)$	$O(1)$

3.7 Similitud entre Transmisiones (Subcadena Común Más Larga)

Nuestra elección: Búsqueda Binaria + Hashing Doble

¿Cómo evitar programar la lenta tabla bidimensional? Aprovechando la *monotonía*. Si existe una subcadena común de longitud L, por fuerza existirá una de longitud menor. Usamos búsqueda binaria sobre la longitud, y comparamos hashes continuos para validar coincidencias. Para que nuestro hash no nos mintiera con falsos positivos, creamos un hashing doble combinando distintos módulos primos; la probabilidad de colisión accidental cayó astronómicamente a 10^{-18}.

Técnica	Tiempo	Espacio
Búsqueda Binaria + Hash	$O((n+m) \log n)$	$O(n + m)$
Tabla DP Dinámica	$O(n \cdot m)$	$O(\min(n,m))$

4. El Octavo Algoritmo (Extra): Propagación y Rutas

Aunque el reto exigía siete módulos, las iteraciones con el profesor nos plantearon una pregunta fascinante: *¿Qué pasa si falla un componente físico en medio de una transmisión?* Para que nuestra interfaz pudiera simular una propagación estilo *Packet Tracer* sin hacer cálculos engañosos en el frontend (JavaScript), tuvimos que implementar en el C++ un motor real de cálculo de caminos mínimos condicionados.

3.8 Modo Extra: Simulación de Enrutamiento de Red

Nuestra elección: Dijkstra (OSPF) y Bellman-Ford (RIP)

Para dotar de realismo a la herramienta, implementamos las bases de dos protocolos clásicos de la vida real. Dijkstra representa algoritmos de estado de enlace (como OSPF) priorizando distancias con una rápida cola de prioridad. Bellman-Ford representa los de vector-distancia (como RIP).

Un desafío monumental fue el **desempate lexicográfico**. Dos rutas pueden tener exactamente el mismo costo de tiempo o saltos, pero para que nuestra animación web no "parpadeara" entre rutas distintas en cada ejecución, forzamos a los algoritmos a decidir empates basándose en la topología (índice del nodo y arista). Gracias a esto, podemos comparar estrictamente que ambos algoritmos devuelvan no solo el mismo costo, sino *exactamente* el mismo árbol de decisión.

Técnica	Tiempo	Espacio
Dijkstra (cola de prioridad)	$O((V+E) \log V)$	$O(V + E)$
Bellman-Ford (con parada)	Peor caso: $O(V E)$	$O(V + E)$

> *Agregamos Bellman-Ford con "parada anticipada": en cuanto una ronda de actualización completa no mejora ninguna distancia en la red, el algoritmo detiene su ejecución inmediatamente, optimizando dramáticamente redes pequeñas.*

5. SENTINEL Digital Twin: Descubrimientos en la Red

Nuestra aplicación "Digital Twin" pasó de ser una simple visualización a un campo de pruebas interactivo. Al interactuar con la red real de GÉANT a través de nuestra plataforma web y jugar con la herramienta de simulación de fallas globales ("¿Qué pasa si falla esto?"), descubrimos fenómenos de red sorprendentes que la teoría topológica simple ocultaba a simple vista:

> ***La fragilidad de Grecia:*** *Descubrimos que Grecia es el eslabón más crítico. De Reino Unido a Grecia caben 80,000 Mbit/s. Si tiramos el enlace con Alemania desde la interfaz, el flujo cae a la mitad. Si tiramos también el enlace con Italia, Grecia queda absolutamente incomunicada (nodos aislados: 1).*

- **El costo del procesamiento de routers importa:** Enviamos un paquete simulado de Eslovenia a Suecia. Si el procesamiento de red es 0 ms, la ruta más rápida da un rodeo inmenso por Europa del Este tomando 6 saltos físicos. Pero si subimos el procesamiento a solo `0.17 ms` por router, la latencia de los saltos se penaliza, y el algoritmo inteligentemente cambia hacia una ruta directa por Austria y Alemania de solo 3 saltos.
- **La red es altamente tolerante a fallas sencillas:** Probamos nuestro analizador heurístico tirando cada uno de los 36 enlaces troncales de forma individual. Impresionantemente, ninguno por sí solo causa aislamientos (la red original no tiene puentes frágiles únicos). La falla que genera mayor retraso general es cortar el cable submarino Nueva York-Reino Unido, incrementando la recepción del mensaje en toda la red de 28.8 ms a 44.6 ms.
- **Diferentes métricas, diferentes decisiones:** Al simular el criterio de selección, vimos empíricamente la diferencia entre OSPF (por distancia) y RIP (por número de saltos). A menudo, en redes europeas densas, usar el conteo bruto de routers (RIP) resultó ser marginalmente más rápido en tiempo final que obsesionarse con los kilómetros de fibra.

6. Correctitud: ¿Cómo validar lo que no conoces?

El mayor obstáculo mental en la ingeniería de algoritmos avanzados es probar sistemas cuando no conoces la "respuesta correcta" esperada en una red de 200,000 nodos. En lugar de confiar en pruebas triviales, implementamos una robusta cadena de validación transversal estructurada en niveles de severidad:

Estrategia	¿Qué comprueba el equipo?
Invariantes Matemáticos	Verificamos que el MST siempre regrese exactamente $n - 1$ aristas; comprobamos la conservación de flujo en cada nodo intermedio (Dinic); y exigimos implacablemente que la suma de las áreas de Voronoi coincida de manera exacta con el área total del canvas de dibujo.
Equivalencia Cruzada	Para cada reto, nuestras dos aproximaciones *deben* coincidir. En cientos de casos aleatorios autogenerados, KMP debía encontrar lo mismo que la fuerza bruta; Dijkstra debía dar el mismo árbol lexicográfico que Bellman-Ford.
Fuerza Bruta Externa	Sometimos al binario en C++ a scripts de validación contra implementaciones puras, ingenuas y seguras redactadas en Python, contrastando ambos binarios contra 300 escenarios simulados intensivos.

Estas validaciones matemáticas nos salvaron innumerables veces. Por ejemplo, la simple verificación de área total en Voronoi nos delató dos errores crípticos de recorte flotante que ninguna prueba visual humana habría detectado en la interfaz gráfica. El estrés de equivalencia cruzada también fue lo que nos obligó a refinar el desempate lexicográfico de Dijkstra detallado anteriormente. Nunca subestimaremos el poder de establecer aserciones matemáticas rígidas a las salidas de nuestros modelos.

7. Mediciones en la Frontera de Rendimiento

Llevamos nuestras dos técnicas de cada problema al extremo. Construimos instancias gigantes con semillas pseudoaleatorias fijas para garantizar la perfecta reproducibilidad de nuestras métricas. Reportamos la mediana de tiempo tras tres corridas, incluyendo la inevitable penalización I/O de leer/escribir el JSON.

Desafío	Instancia de Estrés	Técnica Comparada	Tiempo (ms)	Memoria (MB)
MST	200,000 nodos (1M aristas)	Kruskal Prim	7,179 8,251	1,509 1,509
Flujo Máximo	30,000 nodos (180k aristas)	Dinic Edmonds-Karp	1,122 1,207	274 274
Voronoi	4,000 sitios coordenados	Recorte Directo Vecinos Ordenados	2,024 3,120	66 66
TSP	2,000 puntos (aproximado)	Vecino + Cercano Mejora de 2-opt	60 163	37 37
Firmas	100,000 car. vs 10k pat.	KMP lineal Búsqueda Ingenua	5.4 310.9	4.3 4.4

Calidad en la Aproximación del TSP: Al evaluar los casos donde el exacto era viable (8, 12 y 16 puntos), la mejora 2-opt encontró el óptimo real en el 100% de las veces. En pruebas de estrés masivas aleatorias, descubrimos que su margen de error (desviación respecto a Held-Karp) promediaba un 0.8% extra, y su peor desempeño histórico fue un sobrecosto del apenas 7%. Esta validación numérica nos dio absoluta confianza para usarlo en la red original GÉANT.

8. Teoría contra Práctica: Nuestras 5 Revelaciones

Esta sección encapsula el núcleo humano del proyecto. A menudo, lo que asumes al programar frente al pizarrón se desmorona cuando enciendes el perfilador de hardware. Tres de nuestros cinco mayores hallazgos contradijeron directamente nuestras intuiciones teóricas.

> ***1. El límite del TSP exacto no es el procesador, es la memoria de tu computadora.***
> *Jurábamos que el tiempo estrangularía a Held-Karp. Estábamos equivocados. Al inyectar 16 nodos, el algoritmo computó la respuesta en microscópicos 26 ms, pero la memoria RAM auxiliar consumida se triplicó vertiginosamente. El límite de protección de 16 nodos que impusimos en el código final no es una cifra caprichosa; representa la máxima potencia asintótica manejable antes de que la tabla de subconjuntos hunda las memorias estándar modernas.*

> ***2. Optimizar es a veces empeorar (La ilusión de Voronoi).***
> *Introdujimos un paso de pre-ordenamiento inteligente para los polígonos de Voronoi creyendo que podaríamos recortes inútiles. En la práctica, fue un fracaso espectacular: la versión "optimizada" resultó 54% más lenta. El costo masivo de ejecutar ciclos de ordenamiento* $O(n \log n)$ *por cada sitio evaporó los nanosegundos ahorrados en el recorte geométrico posterior. La lección dolió pero quedó grabada: el algoritmo más ingenuo pero con menor carga de ciclos de máquina suele ser el rey.*

> ***3. Bellman-Ford solo es lento si la red se parece a una cuerda.***
> *Toda la literatura indica que Bellman-Ford pierde por goleada frente a Dijkstra debido a su pesada cota asintótica. Sin embargo, al medir grafos aleatorios de 50,000 nodos, ¡Bellman-Ford ganó el duelo! ¿El secreto? Implementamos un freno anticipado, y en redes aleatorias conexas (que parecen telarañas densas), el camino más largo suele resolverse en poquísimos saltos. Bellman-Ford solo saca a relucir su debilidad cuando modelamos nodos encadenados uno tras otro linealmente (como perlas en un hilo); ahí, el tiempo de proceso se dispara de manera brutal.*

9. Refinando el Sistema: El Choque con la Evaluación

Las iteraciones finales y las charlas con nuestro profesor demostraron que escribir código correcto no siempre significa resolver el problema planteado si no compartes la misma expectativa de entrada/salida.

Un ejemplo punzante fue nuestro modo de ciberseguridad (Firmas). Localmente nuestro algoritmo fallaba al someterse al evaluador oficial, a pesar de usar un KMP brillante. Probamos *once* arquitecturas distintas de formato de salida, convencidos de que teníamos un bug numérico de índices. El profesor nos cedió una guía del formato esperado, y nos dimos cuenta de que nuestro formato JSON era simplemente el equivocado (esperaban una matriz anidada exacta y un `-1` explícito en valores ausentes, en vez de booleanos). Refactorizamos el empaquetado del JSON sin tocar la lógica algorítmica ni una sola línea, y los tests pasaron instantáneamente.

Lección grabada: Dar palos de ciego con formatos de respuesta no escala. Cuando algo falla en producción o en evaluación en línea, no asumas que tu arquitectura interna es pobre; comienza siempre verificando el contrato del protocolo externo.

CONCLUSIÓN DEL EQUIPO

Empezamos *TC2038* resolviendo pedazos aislados de código, pero terminamos construyendo una maquinaria software robusta: Sentinel Digital Twin. Descubrir empíricamente que Kruskal y Prim no tienen un ganador absoluto, o que las strings se devoran por la velocidad y no por la RAM, cambia la mentalidad de programación.

Esta experiencia nos desprendió de la devoción estricta por los cálculos teóricos y nos aterrizó en el arte del monitoreo constante y la ingeniería funcional. Creemos firmemente en lo que hemos entregado y, sobre todo, estamos extremadamente orgullosos del aprendizaje honesto que logramos. Al final del día, los algoritmos viven sobre papel, pero el software real vive en la experimentación y el hardware, sufriendo caídas y requiriendo puentes—exactamente como la red GÉANT.